

LEVEL 3

Chapter 4: Core Building Blocks

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 4: Core Building Blocks

← [Chapter 3: Capacity Estimation](#) · [Index](#) · Next → [Chapter 5: Database Deep Dive](#)

4.1 Load Balancer

Simple explanation: A load balancer sits in front of a pool of identical backend servers and decides, for each incoming request, which server should handle it — so no single server gets overwhelmed and the system can grow by adding more servers behind it.

Analogy: A restaurant host at the door, seeing which of five identical dining rooms has room, and sending the next customer to whichever one keeps things evenly busy — not letting one room overflow while another sits empty.

L4 vs L7

	Layer 4 (Transport)	Layer 7 (Application)
Operates on	IP + port, raw TCP/UDP packets	Full HTTP request — headers, path, cookies, body
Decision-making	"Which server, based on connection info"	"Which server, based on URL path, headers, content" — can route <code>/api/orders</code> to one service and <code>/api/search</code> to another
Speed	Faster (less inspection)	Slower (more inspection), but negligible at typical scale
Example	AWS NLB	AWS ALB, Nginx, Envoy
When to use	Raw TCP/UDP traffic, extreme performance needs, non-HTTP protocols	Any HTTP(S) traffic needing smart routing — the default choice for web/API traffic

Algorithms

Algorithm	How it works	Best for
Round Robin	Requests go to servers in strict rotation (1,2,3,1,2,3...)	Servers with roughly equal capacity and uniform request cost
Least Connections	New request goes to the server with the fewest active connections	Requests with variable processing time (some slow, some fast)
IP Hash	Client IP is hashed to consistently pick the same server	

Algorithm	How it works	Best for
		When you need session affinity without a shared session store (imperfect — NAT/mobile IPs change)
Consistent Hashing	Requests (or cache keys) are hashed onto a ring; a server addition/removal only reshuffles a small fraction of keys	Distributed caches, sharded databases — anywhere minimizing re-distribution on scale-up/down matters (deep dive in Chapter 13)
Weighted Round Robin / Least Connections	Same as above, but servers with more capacity get proportionally more traffic	Heterogeneous server fleet (mixed instance sizes, canary deployments)

Health checks: the load balancer periodically pings each backend (`GET /health`) and stops routing to any instance that fails — this is what actually delivers "failover": a dead instance simply stops receiving new traffic within one health-check interval, typically a few seconds.

Sticky sessions: forcing a client to always hit the same backend instance (via a cookie the LB sets, e.g., `AWSALB`), used when a service is accidentally or intentionally stateful. Prefer externalizing state (Redis-backed sessions) over sticky sessions where you can — sticky sessions complicate deploys (draining a "sticky" instance means kicking users off) and create uneven load if one instance ends up with disproportionately many "sticky" clients.

Scaling the load balancer itself: managed cloud LBs (ALB/NLB, Google Cloud LB) scale automatically and are usually not something you design yourself in an interview — but know that self-hosted LBs (Nginx/HAProxy) need their own redundancy (usually a pair behind a floating/virtual IP, or DNS round-robin across multiple LB nodes) so the load balancer itself isn't a single point of failure.

Interview question: "Your load balancer uses round robin, but you're seeing one backend instance consistently more loaded than the others. Why, and what would you do?"

Ideal senior answer: "Round robin assumes uniform request cost and uniform server capacity — if either assumption breaks, round robin creates imbalance. Common causes: some requests are much more expensive than others (a search query vs. a health check), or the instances aren't actually identical (different instance types, or one is a canary running debug logging). I'd switch to least-connections, which self-corrects for variable request cost, and check whether the instance pool is actually homogeneous."

4.2 CDN (Content Delivery Network)

Simple explanation: A CDN is a globally distributed network of caching servers ("edge locations" or "points of presence") that store copies of your content physically close to users, so a user in Dubai gets content from an edge server in Dubai/nearby instead of your origin server in Mumbai or Virginia.

Why it matters: speed (physical distance = latency, and this is unavoidable — light itself takes ~130ms to cross from India to the US and back), and **origin offload** (your actual servers see far less traffic, since the CDN absorbs the repeated requests for the same content).

Term	Meaning
Edge location	A CDN's physical caching server, located in or near a major population center/ISP
Origin	Your actual server — where content lives when it's not cached at the edge
Cache hit	The edge already has the content — serves it directly, origin never contacted
Cache miss	Edge doesn't have it (or it expired) — CDN fetches from origin, caches it, then serves it
TTL	How long the edge keeps content before re-checking with origin
Cache invalidation	Explicitly telling the CDN "this cached copy is stale, drop it" before the TTL naturally expires (needed when content changes before the TTL window ends)

Static vs. dynamic content: CDNs were built for static assets (images, JS/CSS bundles, videos) that don't change per-request. Modern CDNs (CloudFront, Cloudflare, Fastly) increasingly also cache **dynamic content** intelligently — API responses that are the same for many users (a product listing page), using shorter TTLs and cache-key strategies that account for query params/headers. Content that's genuinely unique per user (your personal order history) shouldn't be cached at a shared CDN layer at all.

Interview question: "You update your product catalog images. How do users see the new image instead of a stale cached one?"

Ideal senior answer: "Two options, and I'd pick based on urgency: either issue an explicit cache invalidation to the CDN — which is an API call, has a small cost, and can take a minute or two to fully propagate globally — or, the pattern I actually prefer for anything non-urgent: change the URL itself when the content changes, e.g., append a content hash or version — `product-123-v2.jpg`. New URL means it's automatically a cache miss, no invalidation call needed, and old cached copies simply expire naturally without anyone needing them anymore. This is exactly what asset bundlers do with hashed filenames for JS/CSS."

(Sharding, edge compute, and CDN failover strategies for a real product are covered inline in Chapter 15/16's problems where relevant, e.g., Netflix, Instagram.)

4.3 Reverse Proxy

Nginx, HAProxy, Envoy are the three you should be able to name and roughly differentiate:

Tool	Primary strength	Common role
Nginx	Fast, simple, huge ecosystem, doubles as a static file server	Reverse proxy, basic load balancing, TLS termination — the default choice for most stacks
HAProxy	Extremely mature, high-performance L4/L7 load balancing	Dedicated load balancer, especially for TCP-level and very high-throughput scenarios
Envoy	Modern, dynamic configuration (no reload needed), deep observability (built-in metrics/tracing), designed for microservices	Service mesh data plane (Istio uses it), API gateway, sidecar proxy

Reverse proxy vs. load balancer: every load balancer is a reverse proxy, but not every reverse proxy is a load balancer. A reverse proxy's core job is simply "sit in front of a server, forward the request, return the response" — it can do this for a single backend (adding TLS termination, compression, caching, or request rewriting along the way) without ever distributing load across multiple servers. A load balancer specifically adds the "which of N identical backends" decision on top of that. In practice, Nginx/Envoy/HAProxy are used for *both* roles simultaneously — reverse-proxying and load-balancing at once — which is why the terms get used almost interchangeably in casual conversation, but you should be able to state the precise distinction if asked.

4.4 API Gateway

Simple explanation: The single front door for all client traffic into your system of (often many) backend services. It's a specialized reverse proxy with extra product-facing responsibilities bolted on.

Responsibility	What it does
Routing	<code>/users/*</code> → user service, <code>/orders/*</code> → order service — one public endpoint, many backends
Authentication	Validates tokens/API keys once, at the edge, so individual services don't each reimplement auth
Rate limiting / throttling	Enforces per-client/per-API-key request limits (token bucket/leaky bucket — Chapter 13) before load even reaches backend services

Responsibility	What it does
Request/response transformation	Reshapes payloads, aggregates multiple backend calls into one client-facing response (a limited form of the "backend for frontend" pattern)
Logging & observability	Centralized point to capture request/response logs and correlation IDs for every request entering the system
Versioning	Can route <code>/v1/orders</code> and <code>/v2/orders</code> to different backend implementations during a migration

Examples: AWS API Gateway (managed), Kong, Apigee, or a self-hosted Envoy/Nginx configured for this role.

Interview question: "Why not just let clients call each microservice directly?"

Ideal senior answer: "A few compounding reasons: clients would need to know the internal topology of your system, which breaks the moment you split or merge a service; every service would need to reimplement auth, rate limiting, and TLS instead of getting it once at the edge; and you lose a single choke point for cross-cutting concerns like request logging and DDoS protection. The trade-off is the gateway becomes a critical dependency and potential bottleneck, so it needs to be highly available and horizontally scalable itself — but that's a much smaller, better-understood problem than distributing all those concerns across every service."

4.5 Caching — Deep Dive

Caching is disproportionately high-value in interviews because almost every design benefits from it, and "just add Redis" without justification is one of the most-cited red flags interviewers report (Chapter 23). This section gives you the *why*, not just the *what*.

Redis vs. Memcached:

	Redis	Memcached
Data structures	Rich: strings, hashes, lists, sets, sorted sets, streams, HyperLogLog, geospatial	Simple key-value strings only
Persistence	Optional (RDB snapshots, AOF log)	None — pure in-memory, lost on restart
Replication/ clustering	Built-in (Redis Cluster, Sentinel for HA)	Client-side sharding only, no built-in replication
Multithreading	Historically single-threaded (I/O threading added in newer versions)	Multi-threaded natively
Extra capabilities	Pub/Sub, Lua scripting, transactions, used as a lightweight queue/rate-limiter	None — deliberately minimal

	Redis	Memcached
When to pick	Default choice for almost everything today — richer feature set, HA story, and can double as a rate limiter or lightweight pub/sub bus	Pure, simple, maximally memory-efficient key-value caching at very large scale with no need for persistence or advanced structures

The four caching patterns

Pattern	How it works	Trade-off
Cache-aside (lazy loading)	App checks cache first; on miss, reads from DB, then writes the result into the cache	Most common pattern. Cache can go briefly stale; first request after expiry is slow ("cold" read)
Read-through	App always talks to the cache; the cache itself is responsible for loading from the DB on a miss (usually via a caching library/proxy)	Simplifies app code (no manual cache-fill logic), but couples the cache layer more tightly to the DB
Write-through	Every write goes to the cache and the DB synchronously, in the same operation	Cache is never stale, but every write pays cache-write latency too
Write-back (write-behind)	Write goes to the cache immediately; the cache asynchronously flushes to the DB later	Fastest writes, but risk of data loss if the cache crashes before flushing — needs careful design for durability
Write-around	Writes go directly to the DB, bypassing the cache entirely; cache only fills on subsequent reads (cache-aside style)	Avoids filling the cache with data that might never be read again, at the cost of a guaranteed-cold first read

Interview question: "Which caching pattern for a social media 'like count'?"

Ideal senior answer: "Cache-aside for reads combined with write-through (or even write-back) for the counter itself — likes are extremely read-heavy relative to writes and users tolerate a few seconds of staleness on a like count, so I'd actually increment the counter directly in Redis (using **INCR**, which is atomic) as the source of truth for the hot path, and asynchronously persist to the database periodically or via a write-behind queue, rather than hitting the DB synchronously on every single like."

TTL and Eviction

TTL (time-to-live): every cached entry should have an expiration — caches with no TTL policy silently grow stale forever or grow unbounded in memory. Choose TTL based on how quickly the underlying data actually changes and how costly staleness is (a product price: short TTL; a user's display name: longer TTL is usually fine).

Eviction policies — what happens when the cache is full and a new item needs space:

Policy	Rule	Best for
LRU (Least Recently Used)	Evict the item that hasn't been accessed in the longest time	General-purpose default — Redis's default when <code>maxmemory-policy</code> is set
LFU (Least Frequently Used)	Evict the item accessed the fewest times overall	Workloads where "popular" items should stay cached even if not accessed <i>recently</i> (e.g., a perennially popular product vs. one that had one burst of traffic)
FIFO	Evict the oldest inserted item, regardless of usage	Rarely ideal — usually a fallback, not a deliberate choice
TTL-based	Evict purely by expiration time, independent of access pattern	Time-sensitive data (session tokens, OTPs)

The three cache failure modes interviewers specifically probe for

Problem	What happens	Fix
Cache stampede (a.k.a. thundering herd)	A hot key expires; thousands of concurrent requests all miss at once and hammer the DB simultaneously trying to refill it	Locking (only one request refills, others wait briefly) or probabilistic early expiration (refresh slightly before actual TTL, staggered) or request coalescing (in-flight de-duplication)
Cache penetration	Requests for keys that don't exist in the cache <i>or</i> the DB (e.g., an invalid/malicious ID) bypass the cache every time and always hit the DB	Cache the "not found" result too (with a short TTL), or use a Bloom filter to cheaply reject known-nonexistent keys before even querying
Cache avalanche	A large number of keys expire at the same moment (e.g., all cached at once during a deploy with identical TTLs), causing a simultaneous mass cache-miss and a DB traffic spike	Jitter the TTLs (add random variance so keys don't all expire simultaneously) and ensure the DB/backing store can survive a partial-cache-miss burst

Hot keys: a small number of keys (a viral post, a celebrity's profile, a flash-sale product) receive disproportionate traffic — so disproportionate that even a single Redis node/shard responsible for that key becomes a bottleneck, no matter how well the rest of the cache is distributed. Fixes: **local (in-process) caching** of the hottest keys on each app server as a first line of defense, **key replication** (store the same hot key on multiple cache nodes and randomly pick one to read), or splitting a hot counter into N sub-counters across shards that get summed on read.

Distributed cache & invalidation: once your cache spans multiple nodes, you need consistent hashing (Chapter 5/13) to decide which node owns which key, and — the genuinely hard part — invalidating a key across all nodes/regions consistently when the underlying data changes, especially in multi-region setups (usually solved with **pub/sub-based invalidation broadcasts** or accepting bounded staleness).

4.6 Databases — The Selection Question

This is a preview; Chapter 5 goes deep on internals. Here, focus on the decision framework, because "how do I choose a database" is asked in some form in nearly every HLD interview.

SQL

MySQL, PostgreSQL — relational, schema-enforced, **ACID**-compliant (Atomicity, Consistency, Isolation, Durability — a transaction either fully happens or not at all, and the DB enforces your integrity rules). Strong for anything with relationships that need to be queried flexibly and data that must never violate integrity rules (a payment can't reference a nonexistent order).

NoSQL

Type	Examples	Data model	Best for
Key-Value	Redis, DynamoDB (can also do more)	Simple key → value	Ultra-fast lookups by a known key: sessions, caching, feature flags
Document	MongoDB	JSON-like documents, flexible schema	Semi-structured data with varying shape per record, rapid iteration (e.g., product catalogs with wildly different attributes per category)
Wide-column	Cassandra, HBase	Rows with dynamic columns, partitioned by a row key	Massive write throughput, time-series-ish data, need for multi-datacenter active-active writes
Managed key-value/document (cloud-native)	DynamoDB	Key-value + document hybrid, fully managed	Need predictable single-digit-ms latency at any scale without operating the database yourself

The decision framework you say out loud in an interview

Ask yourself, in order:

1. **Do I need multi-record ACID transactions and complex relational queries (joins, aggregations across entities)?** → SQL.
2. **Is my schema going to vary a lot per record, or evolve rapidly, and do I mostly fetch whole documents by ID?** → Document DB (MongoDB).
3. **Do I need to sustain extremely high write throughput, ideally across multiple regions/datacenters simultaneously, and my query patterns are simple and known in advance (fetch by partition key)?** → Wide-column (Cassandra) or DynamoDB.
4. **Do I just need blazing-fast key lookups, and I can tolerate losing the data on a crash (or it's derived/reconstructible)?** → Redis/key-value.
5. **Am I unsure and the data is genuinely relational with moderate scale?** → Default to SQL. It's the safer, more battle-tested default, and "premature NoSQL" is as real a mistake as premature microservices.

Requirement	Choose	Why
Bank account balances, order/payment ledger	PostgreSQL/MySQL	ACID transactions non-negotiable
Product catalog with wildly varying attributes per category	MongoDB	Flexible schema, document-per-product
Chat message history at massive write scale	Cassandra	Write-optimized, partition by conversation ID
Session store / rate limiter counters	Redis	Sub-millisecond, in-memory, TTL support
User profile lookups at global scale, predictable latency	DynamoDB	Fully managed, auto-scaling, single-digit ms
Full-text product/content search	Elasticsearch (alongside, not instead of, your primary DB)	Inverted index built for text relevance ranking — see Chapter 22

Interview question: "Why not just use MongoDB for everything and avoid this decision entirely?"

Ideal senior answer: "Because 'flexible schema' is a feature until it's a liability — without enforced structure, you eventually build the validation and consistency logic that a relational DB gives you for free, just in application code, badly, across N places. And MongoDB's multi-document transactions, while they exist now, aren't as natural a fit for deeply relational, integrity-critical data as a mature RDBMS. I'd rather pick the database that matches the *shape and integrity requirements* of each specific piece of data — which is very often exactly why real systems at scale run multiple database technologies side by side (polyglot persistence), not one database for everything."

Chapter 4 Interview Drill

1. State the precise difference between a reverse proxy and a load balancer.
2. Walk through what happens on a CDN cache miss, start to finish.
3. Name all three cache failure modes (stampede, penetration, avalanche) and their fixes, without looking.
4. Give a concrete example of a hot key problem and two ways to fix it.
5. Walk through your SQL-vs-NoSQL decision tree for: (a) a payments ledger, (b) a product catalog, (c) a chat history store.

Next → [Chapter 5: Database Deep Dive](#) — indexes, transactions, replication, and the scaling ladder from one database to a distributed one.